

Maze I (LeetCode 490)

Comprehensive Notes & C++ Solution

© Authored by Arpit Raj, LNMIIT Jaipur | July 23, 2026

1. Problem Overview

Given a maze represented by a grid where:

- 0 = Empty space
- 1 = Wall

Aadya starts at `start = [sr, sc]` and wants to reach `destination = [dr, dc]`.

Question: Can Aadya stop exactly at the destination?
Notice: It is NOT enough to just "pass through" the destination. She **must stop there**.

2. The Twist: Special Rule

Unlike previous grid problems, Aadya cannot stop whenever she wants. This is the whole twist.

She chooses a direction (**Up, Down, Left, Right**) and keeps rolling until she hits:

- A wall (1)
- The boundary of the maze

Only then does she stop, and only then can she choose another direction.

3. Dry Run Example

Suppose we have the following maze, and Aadya starts at `(2, 0)`.

0	0	0	1
0	1	0	0
A	*	S	X

■ Start ■ Roll Path ■ Stopping Point ■ Wall (1)

Rolling Right:

- Start at $(2, 0)$.
- Moves right to $(2, 1) \rightarrow$ *DOES NOT stop here.*
- Moves right to $(2, 2) \rightarrow$ *DOES NOT stop here yet.*
- Next cell $(2, 3)$ is a wall $(X / 1)$.
- She finally stops at $(2, 2)$.

4. Graph Interpretation & Complexity

In standard grid traversal:

- **Vertex** = Cell
- **Edge** = Adjacent Cell

In this Maze problem:

Vertex = Stopping Position

Edge = Roll until wall

Why BFS Still Works?

Each roll (edge) has equal cost (Roll Left = 1 move, Roll Right = 1 move). We are not minimizing the rolling distance; we only care about reachability. Hence BFS (or DFS) is perfectly valid.

Time Complexity

Suppose the maze is $n \times m$. There are at most $n \times m$ stopping positions to visit. For each visited position, rolling in one direction may travel up to $O(\max(n, m))$ cells.

Therefore, the Time Complexity is: $O(n \times m \times (n + m))$

5. C++ Solution

```
class Solution {
public:

    vector<pair<int,int>> dir={{-1,0},{0,-1},{1,0},{0,1}};

    bool hasPath(vector<vector<int>>& maze, vector<int>& start, vector<int>&
destination) {
```

```

int n=maze.size();
int m=maze[0].size();

queue<pair<int,int>> q;
vector<vector<int>> seen(n,vector<int>(m,0));

// Aadya starts from the given start position.
q.push({start[0],start[1]});
seen[start[0]][start[1]]=1;

while(!q.empty()){
    int currsz=q.size();
    while(currsz--){

        int r=q.front().first;
        int c=q.front().second;
        q.pop();

        // IMPORTANT:
        // We only check after popping because every node in the queue
        // represents a VALID STOPPING POSITION.
        // If Aadya reaches destination while rolling,
        // it is NOT enough.
        // She must actually stop there.
        if(r==destination[0] && c==destination[1]){
            return true;
        }

        for(auto it:dir){

            // Start rolling from current stopping position.
            int nr=r;
            int nc=c;

            // Keep rolling until next move becomes invalid.
            // We check the NEXT cell instead of current cell.

            // Aadya should stop at the last '.'
            // not on the wall.
            while(nr+it.first>=0 && nr+it.first<n &&
                nc+it.second>=0 && nc+it.second<m &&
                maze[nr+it.first][nc+it.second]==0){

                nr+=it.first;
                nc+=it.second;
            }

            // (nr,nc) is now the stopping position.
            // If we have never reached this stopping position before,
            // continue BFS from here.
            if(!seen[nr][nc]){
                seen[nr][nc]=1;
                q.push({nr,nc});
            }
        }
    }
}

```

```
        }  
    }  
  
    // Explored every reachable stopping position.  
    // Destination was never reached.  
    return false;  
}  
};
```